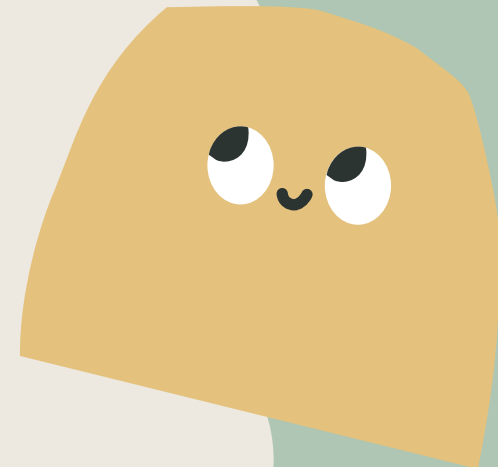
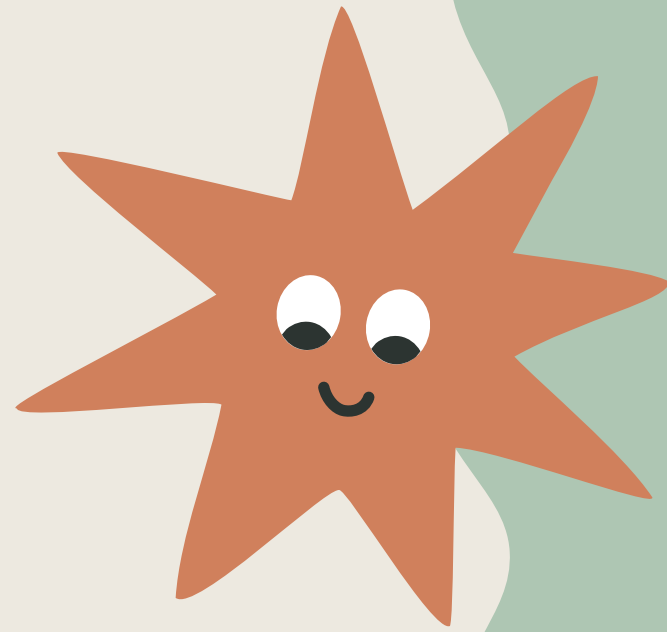


Stack — Concept, Operations, Applications

Lecture 12



Learning Outcomes (Lecture 12)

➤ At the end of this lecture, students will be able to:

- Define a Stack and the **LIFO** principle
- Understand Stack as an **Abstract Data Type (ADT)**
- Explain the concept of TOP
- Perform
 - Push, Pop, Peek/Top, isEmpty, isFull
- Understand **Overflow** and **Underflow**
- Write algorithms for basic Stack operations.
- Analyze their time complexity.
- Explain practical applications of Stack.
- Understand how a Stack can be implemented using arrays and linked lists

What is Stack?

- A Stack is a linear Abstract Data Type in which insertion and deletion are performed from only one end
- A **stack** is a linear data structure that follows the **Last In, First Out (LIFO)** principle
- This means, last element inserted would be the first to be deleted

STACK

A stack is a linear data structure that follows the **LIFO** principle (**Last In, First Out**).

DEFINITION

A **stack** is a linear data structure in which insertion and deletion of elements are done from one end only, called the **TOP**.

ANALOGY

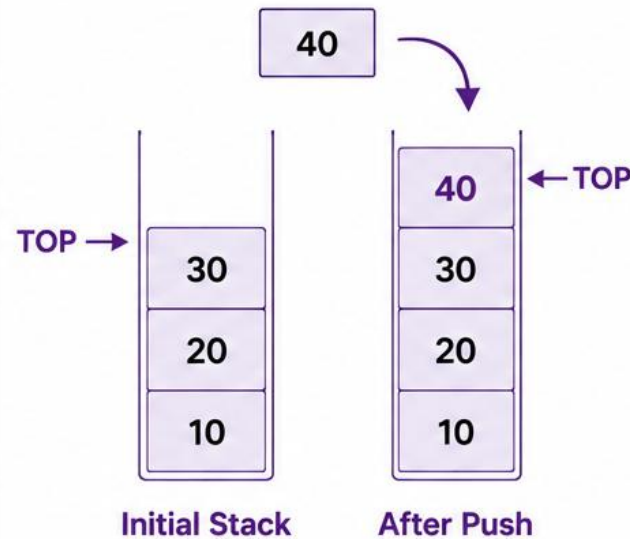
A stack is like a stack of plates.

- We place a plate on top.
- We remove a plate from the top.
- We cannot remove a plate from the bottom directly.



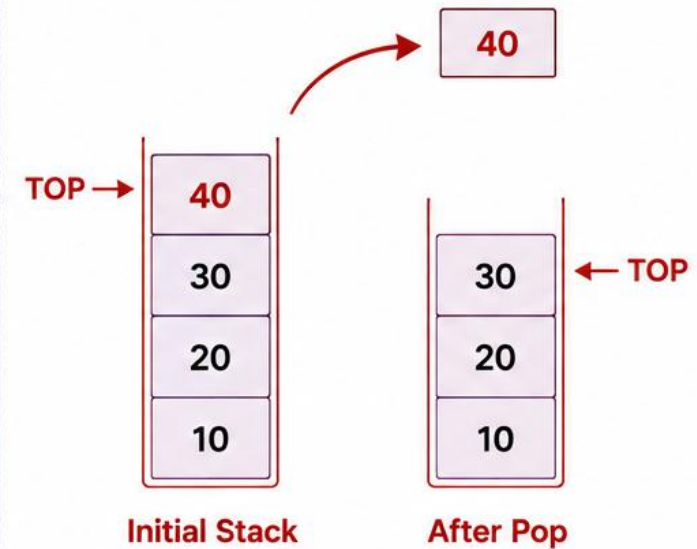
PUSH (Insert)

A new element is added at the **TOP**.



POP (Delete)

The **top** element is removed from the stack.



Key Point: New elements are added only at the **TOP**.
Elements are removed only from the **TOP**.

This is why stack follows **LIFO (Last In, First Out)**.

Important Terms

➤ Top

- TOP identifies the element currently available at the top of the Stack

➤ Stack Overflow

- Occurs when trying to push an item into a completely full stack
- Overflow is particularly relevant to fixed-size array implementations

➤ Stack Underflow

- Occurs when trying to pop or peek an item from an empty stack

Applications of Stack

➤ Function Calls

- The system maintains information about function calls using a **call stack**

➤ Recursion

- The recursive calls are managed using the **call stack**

➤ Undo Operation

- Undo with MS Word, Photoshop, Code editors etc

➤ Browser Back

- A Stack can be used to manage browser history

➤ Expression Evaluation

- Stack helps evaluate the expression

Basic Operations on Stack

➤ A stack has following primary operations:

- Push – Adds an element to the top of the stack
- Pop – Removes and returns the top element
- Peek (or Top) – Views the top element without removing it
- isEmpty – Checks if the stack has no elements
- isFull – Checks if a fixed-size stack has reached capacity

STACK – PUSH OPERATION

Push operation is used to **insert** a new element at the **TOP** of the stack.

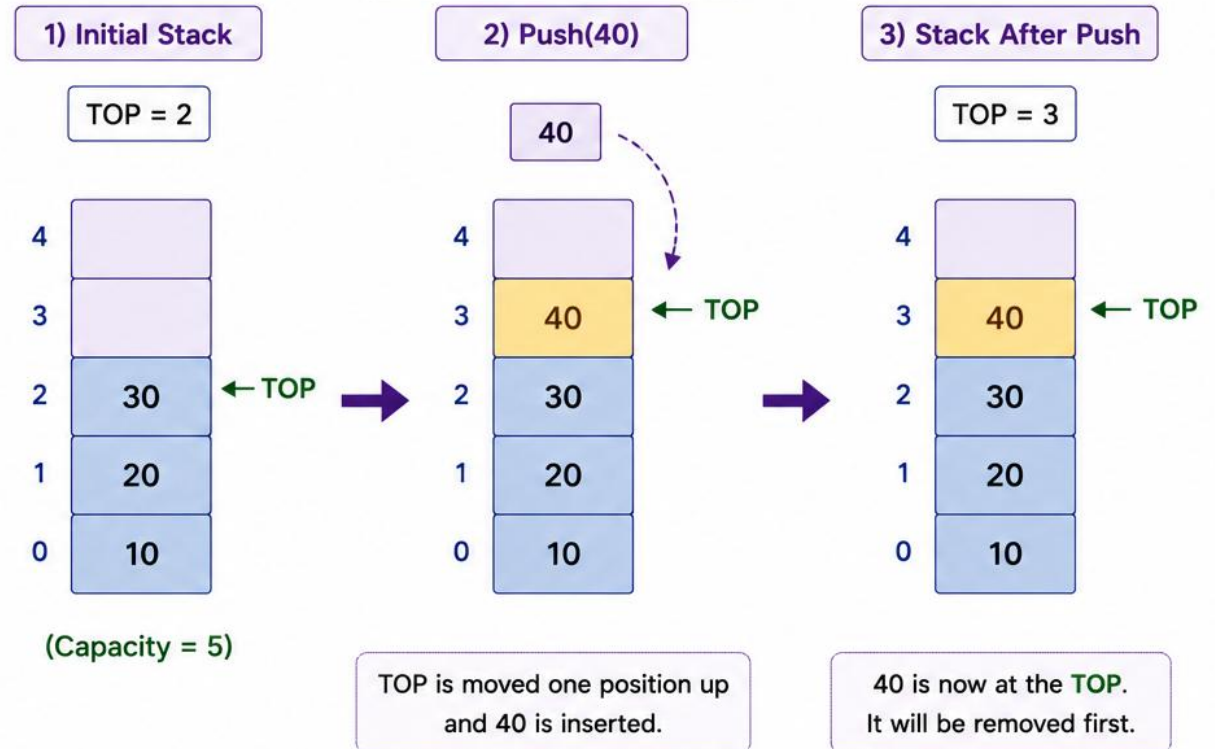
ALGORITHM

- 1 Check if the stack is full.
 - If full, report **Overflow** and stop.
- 2 Otherwise,
 - Move **TOP** one position up.
 - Insert the new element at **TOP**.
- 3 Stop.

EXPLANATION

- New element always goes on the **TOP**.
- After pushing, **TOP** points to the newly inserted element.
- Push operation follows **LIFO** principle (**Last In, First Out**).

VISUAL EXPLANATION



TIME COMPLEXITY

Push Operation: **O(1)**

Push operation takes constant time because we only move **TOP** once and insert the element.

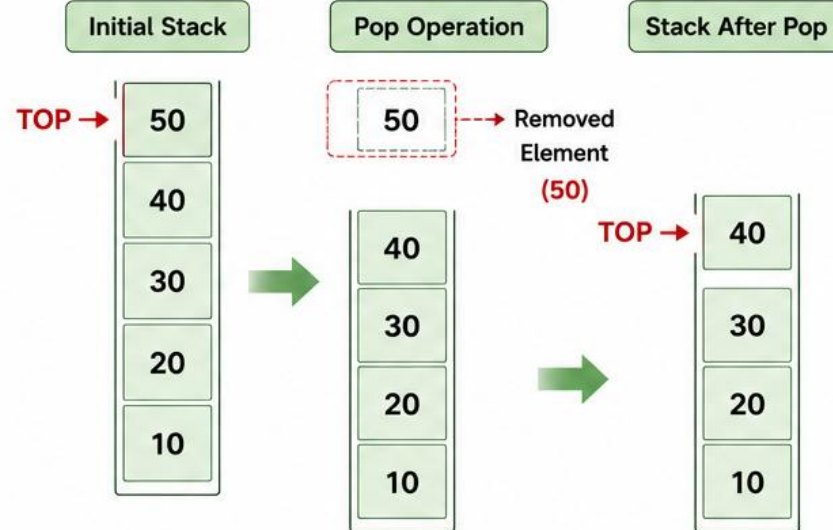
STACK – POP OPERATION

Pop operation removes the **top** element from the stack and returns it.

WHAT IS POP?

- The pop operation deletes the element present at the **TOP**.
- After deletion, **TOP** is moved to the next element.
- The removed element is returned as the result of pop.

EXAMPLE



ALGORITHM (POP)

- 1 Check if the stack is empty.
If yes, then
→ **Underflow** condition occurs.
Stop the operation.
- 2 Otherwise,
store the element at **TOP**
in a temporary variable
(say, item).
- 3 Decrease **TOP** by 1.
- 4 Return the stored element (item)
as the popped element.

IMPORTANT NOTES

- Pop always works on the **TOP** element.
- After pop, the previous element becomes the new **TOP**.
- If the stack is empty and pop is performed, it is called **UNDERFLOW**.

GENERAL TIME COMPLEXITY



$O(1)$

Pop operation takes constant time because it performs a fixed number of simple steps.



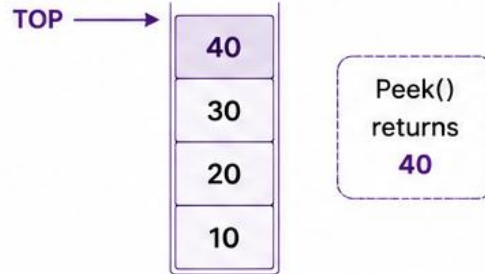
Key Point: **Pop** removes the **top** element in constant time and makes the previous element the new **top**.

STACK OPERATIONS (Peek, IsEmpty, IsFull)

1. PEEK (Top)

Peek (or Top) returns the top element of the stack **without** removing it.

EXAMPLE



ALGORITHM

1. If stack is empty
→ **Underflow** (no element to return).
2. Otherwise
→ Return the element at **TOP**.
3. Do not modify **TOP**.
4. Stop.

TIME COMPLEXITY



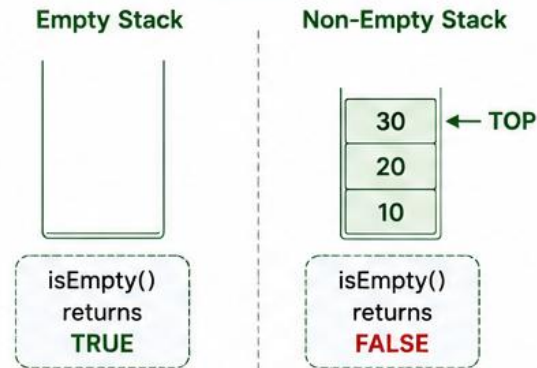
O(1)

Constant Time

2. ISEMPY

isEmpty() checks whether the stack **does not** contain any element.

EXAMPLE



ALGORITHM

1. If **TOP** = -1 (stack is empty)
→ Return **TRUE**.
2. Otherwise
→ Return **FALSE**.
3. Stop.

TIME COMPLEXITY



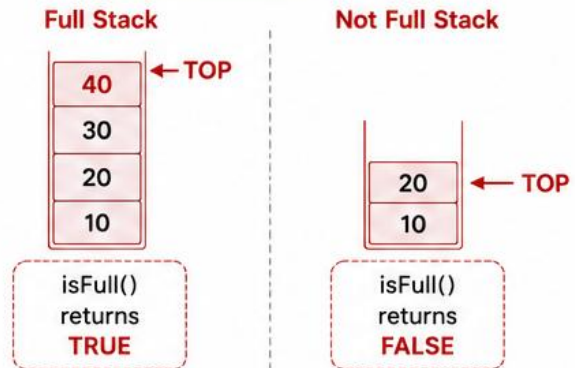
O(1)

Constant Time

3. ISFULL

isFull() checks whether the stack has **no space** to insert new element.

EXAMPLE (Capacity = 4)



ALGORITHM

1. If **TOP** = **Capacity - 1** (stack is full)
→ Return **TRUE**.
2. Otherwise
→ Return **FALSE**.
3. Stop.

TIME COMPLEXITY



O(1)

Constant Time



Key Takeaway:

Peek gives the top element **without** removing it.
isEmpty tells whether stack is empty.

isFull tells whether stack is full (based on capacity).